What is JavaScript? Explain the role of JavaScript in web development.

->JavaScript is a high-level, interpreted programming language primarily used to create interactive

and dynamic content on websites.

->It runs in the browser and allows developers to manipulate web pages in real-time.

Role in web development:

- Manipulates HTML and CSS dynamically using the DOM

- Handles events like clicks, typing, and page loads

- Enables communication with servers using AJAX.

- Forms the core of front-end frameworks like React, Angular etc.

How is JavaScript different from other programming languages like Python or Java?

JavaScript differs from Python and Java in several ways:

- Environment: JavaScript mainly runs in web browsers, while Python and Java run on servers or standalone applications.

- Typing: JavaScript is dynamically typed, whereas Java is statically typed.

- Compilation: JavaScript is interpreted (just-in-time compiled), while Java is compiled to bytecode.

- Concurrency Model: JavaScript uses an event-driven, non-blocking model; Python and Java use multi-threading.

- Use: JavaScript is mainly for web development, Python is used in data science, AI, scripting, and Java is used in enterprise applications.

Discuss the use of <script> tag in HTML. How can you link an external JavaScript file to an HTML document?

The <script> tag in HTML is used to include JavaScript code within a webpage. It can contain inline JavaScript or link to an external file.

Inline Example:

<script>

console.log('Hello World');

</script>

External File Linking:

<script src='script.js'></script>

What are variables in JavaScript? How do you declare a variable using var, let, and const?

Variables in JavaScript are used to store data values. They act as containers for storing information that can be used and manipulated in a program.

Declaration:

- **var**: Function-scoped and can be redeclared.

Example: var x = 10;

- **let**: Block-scoped and cannot be redeclared in the same scope.

Example: let y = 20;

- **const**: Block-scoped and used for constants (cannot be reassigned).

Example: const z = 30;

Key Difference:

- var is older and less preferred.

- let is used when value may change.

- const is used when value should remain constant.

Explain the different data types in JavaScript. Provide examples for each.

JavaScript has several data types:

1. Primitive Data Types:

- Number: let a = 10;

- String: let name = 'John';

- Boolean: let isActive = true;

- Undefined: let x;

- Null: let y = null;

2. Non-Primitive (Reference) Data Types:

- Object: let obj = {name: 'John', age: 25};

- Array: let arr = [1, 2, 3];

- Function: function greet() { return 'Hello'; }

What is the difference between undefined and null in JavaScript?

Undefined and null both represent absence of value but are different:

- **undefined**: A variable has been declared but not assigned a value.

Example: let a;

- **null**: Represents an intentional absence of value (assigned by developer).

Example: let b = null;

Differences:

- undefined is automatically assigned by JavaScript.

- null is explicitly assigned.

- Type of undefined is 'undefined'.

- Type of null is 'object' .

What are the different types of operators in JavaScript? Explain with examples.

Operators are symbols used to perform operations on variables and values.

1. Arithmetic Operators

Used for mathematical calculations:

- + (Addition): 5 + 3 = 8
- - (Subtraction): 5 - 3 = 2
- * (Multiplication): 5 * 3 = 15
- / (Division): 6 / 2 = 3
- % (Modulus): 5 % 2 = 1

2. Assignment Operators

Used to assign values to variables:

- = : x = 10
- += : x += 5 → x = x + 5
- -= : x -= 5
- *= : x *= 2
- /= : x /= 2

3. Comparison Operators

Used to compare values (returns true or false):

- == : 5 == '5' → true
- === : 5 === '5' → false
- != : 5 != 3 → true
- > : 5 > 3 → true
- < : 5 < 3 → false
- >= : 5 >= 5 → true
- <= : 5 <= 3 → false

4. Logical Operators

Used to combine conditions:

- && (AND): true && false → false
- || (OR): true || false → true
- ! (NOT): !true → false

What is the difference between == and === in JavaScript?

“==” Compares values after type conversion

Example:

5 == '5' // true

“===” Compares both value and data type

No type conversion

Example:

5 === '5' // false

What is control flow in JavaScript? Explain how if-else statements work with an example.

Control flow refers to the order in which the statements in a program are executed. It allows the program to make decisions and run different code based on conditions.

if-else Statement

The if-else statement is used to execute code based on a condition.

Syntax:

```
if (condition) {
// code runs if condition is true
} else {
// code runs if condition is false
}
```

Example:

```
let age = 18;
if (age >= 18) {
console.log("You are eligible to vote");
} else {
console.log("You are not eligible to vote");
}
```

- If the condition (age >= 18) is true, the if block runs
- If the condition is false, the else block runs

Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

A switch statement is used to perform different actions based on different values of a variable.

Syntax:

```
switch(expression) {
case value1:
// code
```

```
break;
case value2:
// code
break;
default:
// default code
}
```

Example:

```
let day = 2;
switch(day) {
case 1:
console.log("Monday");
break;
case 2:
console.log("Tuesday");
break;
default:
console.log("Other day");
}
```

Working:

- The switch checks the value of day
- It matches the case (2) and runs that block
- break stops further execution
- default runs if no case matches

When to use switch instead of if-else:-

Use switch when:

- You are checking one variable against many fixed values
- Code becomes cleaner and easier to read

Use if-else when:

- You have complex conditions (e.g., ranges like age > 18)
- Multiple variables or logical expressions are involved

Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

for loop:

Used when the number of iterations is known.

Example:

```
for (let i = 0; i < 5; i++) {
console.log(i);
}
```

while loop:

Executes as long as the condition is true.

Example:

```
let i = 0;
while (i < 5) {
console.log(i);
i++;
}
```

do-while loop:

Executes at least once, then checks condition.

Example:

```
let i = 0;
do {
console.log(i);
i++;
} while (i < 5);
```

What is the difference between a while loop and a do-while loop?

while loop:

- Condition is checked BEFORE execution

- May not execute even once

Example:

```
let i = 10;
while (i < 5) {
console.log(i);
}
```

do-while loop:

- Condition is checked AFTER execution

- Runs at least once

Example:

```
let i = 10;
do {
console.log(i);
} while (i < 5);
```

What are functions in JavaScript? Explain the syntax for declaring and calling a function.

->Functions in JavaScript are reusable blocks of code designed to perform a specific task.

->They help in organizing code, improving readability, and avoiding repetition.

Syntax (Function Declaration):

```
function functionName(parameters) {
// code to be executed
}
```

Calling a Function:

functionName(arguments);

Example:

```
function add(a, b) {
return a + b;
}
add(2, 3); // Output: 5
```

What is the difference between a function declaration and a function expression?

Function Declaration:

->Defined using the function keyword.

->Hoisted (can be called before it is defined in the code).

Example:

```
function greet() {
console.log("Hello");
}
```

Function Expression:

->Function is assigned to a variable.

->Not hoisted (cannot be used before declaration).

Example:

```
const greet = function() {
console.log("Hello");
};
```

Discuss the concept of parameters and return values in functions.

Parameters:

->Parameters are variables listed in the function definition.

->They act as inputs to the function.

Example:

```
function multiply(a, b) {
return a * b;
}
```

Return Values:

A return value is the output that a function sends back using the return statement.

Example:

```
let result = multiply(2, 3); // result = 6
```

What is an array in JavaScript? How do you declare and initialize an array?

An array in JavaScript is a special variable used to store multiple values in a single variable. These values can be of any data type (numbers, strings, objects, etc.).

Declaring and Initializing an Array:

Method 1: Using square brackets

```
let arr = [1, 2, 3, 4];
```

Method 2: Using Array constructor

```
let arr = new Array(1, 2, 3, 4);
```

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
```

Explain the methods push(), pop(), shift(), and unshift() used in arrays.

These are commonly used array methods to add or remove elements.

1. push()

- Adds element(s) to the end of the array.

```
let arr = [1, 2];
arr.push(3); // [1, 2, 3]
```

2. pop()

- Removes the last element from the array.

```
let arr = [1, 2, 3];
arr.pop(); // [1, 2]
```

3. shift()

- Removes the first element from the array.

```
let arr = [1, 2, 3];
arr.shift(); // [2, 3]
```

4. unshift()

- Adds element(s) to the beginning of the array.

```
let arr = [2, 3];
arr.unshift(1); // [1, 2, 3]
```

What is an object in JavaScript? How are objects different from arrays?

->An object in JavaScript is a collection of key-value pairs, where each key (also called a property) is associated with a value.

->Objects are used to store related data and functionality together.

Example:

```
let person = {
  name: "John",
  age: 25,
```

```
  city: "Delhi"
};
```

Difference between Objects and Arrays:

Object	Array
Stores data as key-value pairs	Stores data as ordered list
Keys are used to access values	Index (0,1,2...) is used
Unordered (no fixed position)	Ordered (fixed positions)
Example: {name: "John"}	Example: ["John", 25]

Explain how to access and update object properties using dot notation and bracket notation.

There are two ways to access and update object properties:

1. Dot Notation (.)

- Used when property name is known and simple.

Access:

```
let person = { name: "John", age: 25 };
console.log(person.name); // John
```

Update:

```
person.age = 30;
```

2. Bracket Notation ([])

- Used when property name is dynamic or contains spaces.

Access:

```
console.log(person["name"]); // John
```

Update:

```
person["age"] = 35;
```

What are JavaScript events? Explain the role of event listeners.

->JavaScript events are actions or occurrences that happen in the browser, such as a user clicking a button, pressing a key, moving the mouse, or loading a page.

->Events allow JavaScript to respond to user interactions and make web pages interactive.

Examples of events:

- click
- keydown
- mouseover
- submit

Role of Event Listeners:
Event listeners are used to detect when an event occurs and execute a specific function (called a callback function) in response.

How does the addEventListener() method work in JavaScript? Provide an example.

The addEventListener() method is used to attach an event handler to an element without overwriting existing event handlers.

Syntax:

element.addEventListener("event", function);

- "event" → Name of the event (e.g., "click")
- function → Function to execute when the event occurs

Example:

```
let btn = document.getElementById("myButton");

btn.addEventListener("click", function() {
  alert("Button clicked!");
});
```

Explanation:

- When the button is clicked, the function runs.
- It shows an alert message.

What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

->The DOM (Document Object Model) is a programming interface for web documents.

->It represents the structure of an HTML document as a tree of objects, where each element (like , <p>, <h1>) is a node.

->JavaScript uses the DOM to access, modify, and manipulate HTML and CSS dynamically.

How JavaScript interacts with the DOM:

- Select elements
- Change content (text, HTML)
- Modify styles (CSS)
- Add or remove elements
- Handle events (click, submit, etc.)

Example:

```
document.getElementById("demo").innerHTML = "Hello World!";
```

Explain the methods getElementById(), getElementsByClassName(), and querySelector() used to select elements from the DOM.

These methods are used to select HTML elements so JavaScript can work with them.

1. getElementById()

- Selects a single element by its unique ID.

Example:

```
let element = document.getElementById("myId");
```

2. getElementsByClassName()

- Selects multiple elements that have the same class name.
- Returns a collection (array-like object).

Example:

```
let elements = document.getElementsByClassName("myClass");
```

3. querySelector()

- Selects the first element that matches a CSS selector.

Example:

```
let element = document.querySelector(".myClass");
```

Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?

setTimeout() and setInterval() are JavaScript functions used to execute code after a certain delay or repeatedly at fixed time intervals.

setTimeout():

- Executes a function once after a specified delay (in milliseconds).

Syntax:

```
setTimeout(function, delay);
```

setInterval():

- Executes a function repeatedly at a specified interval (in milliseconds).

Syntax:

```
setInterval(function, interval);
```

Usage in Timing Events:

- Used for animations, notifications, delays, auto-refresh, etc.
- Helps control when and how often code runs.

Example:

```
setInterval(function() {
  console.log("Runs every 1 second");
}, 1000);
```

Provide an example of how to use setTimeout() to delay an action by 2 seconds.

Example 1:-

```
setTimeout(function() {
  console.log("This message appears after 2 seconds");
}, 2000);
```

Example 2:-

```
function showMessage() {
alert("Hello after 2 seconds!");
}

setTimeout(showMessage, 2000);
```

What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.

->Error handling in JavaScript is a way to manage and respond to runtime errors so that the program does not crash and can handle unexpected situations gracefully.

->JavaScript provides try, catch, and finally blocks for handling errors.

try block:

- Contains code that may produce an error.

catch block:

- Executes if an error occurs in the try block.
- Handles the error.

finally block:

- Always executes, whether an error occurs or not.

Syntax:

```
try {
  // code that may cause error
} catch (error) {
  // code to handle error
} finally {
```

```
  // code that always runs
}
```

Example:

```
try {
  let x = 10;
  let y = x + z; // z is not defined (error)
} catch (error) {
  console.log("An error occurred:", error.message);
} finally {
  console.log("This will always run");
}
```

Why is error handling important in JavaScript applications?

Error handling is important because:

- **Prevents application crashes** – Keeps the program running even if errors occur.
- **Improves user experience** – Shows meaningful messages instead of breaking the app.
- **Helps debugging** – Makes it easier to identify and fix issues.
- **Ensures smooth execution** – Handles unexpected situations properly.
- **Maintains code reliability** – Makes applications more stable and robust.